# Metode de rezolvare a recurențelor

October 15, 2024

## Contents


**1 Sortarea prin interclasare (Merge-Sort)** **1**

**2 Metoda substituției** **2**
2.1 Prezentare . . . . . 2
2.2 Aplicarea metodei pentru Merge-Sort . . . . . 2

**3 Teorema Master** **4**
3.1 Enunțul teoremei . . . . . 4
3.2 Math cheatsheet . . . . . 4

**4 Exerciții** **5**

**5 Anexe** **6**
5.1 Completarea demonstrației analizei complexității Merge-Sort . . . . . 6


# 1 Sortarea prin interclasare (Merge-Sort)

**Algorithm 1** Pseudocod Merge-Sort

```
1: function MERGE-SORT(A, p, r)
2:     if p ≤ r then
3:         q = ⌊(p + r)/2⌋
4:         MERGE-SORT(A, p, q)
5:         MERGE-SORT(A, q + 1, r)
6:         MERGE(A, p, q, r)
7:     end if
8: end function
```

Pentru a simplifica analiza complexității pseudocodul 1, vom presupune că $n$, dimensiunea problemei, este o putere a lui 2 [1, cap. 2.3.2, subcap. *Analysis of merge sort*]. (algoritmul funcționează corect pentru orice $n$). Recurența pentru Merge-Sort se poate rezolva ușor dacă se observă că problema inițală $T(n)$ se împarte in două subprobleme, $T(\frac{n}{2})$, la a căror complexitate se adună cea pentru `Merge()`. Funcția `Merge()` are complexitatea $\Theta(n)$ deoarece parcurge elementele celor doi vectori o singură dată. Rezultă că Merge-Sort are recurența de forma:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

Folosind metoda iterației sau cea a arborilor de recurență, rezultă $T(n) \in \Theta(n \cdot log(n))$.

# 2 Metoda substituției

## 2.1 Prezentare

Metoda substituției se folosește atunci cand este posibilă intuirea complexității unei probleme care are atașată o relație de recurență. După ce se găsește soluția, aceasta trebuie verificată folosind inducția matematică. Schema pentru metoda substituției este:

- alegem soluția:
  - știm complexitatea pentru o recurență asemănătoare
  - stabilim limite inferioare și superioare din ce in ce mai apropiate de soluție
  - se folosesc celelalte metode studiate
- verificăm apartenența lui $T(n)$ la o anumită clasă de complexitate prin inducție:
  - caz de bază
  - pas de inducție de tipul $P(n) \Rightarrow P(n+1)$ sau $P(n/k) \Rightarrow P(n)$

## 2.2 Aplicarea metodei pentru Merge-Sort

Ne propunem să oferim o demonstrație formală a faptului că algoritmul Merge-Sort are complexitatea $\Theta(n \cdot log(n))$. Fie:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n, T(1) = 1$$

Vrem să demonstrăm că există două constante $c_1, c_2 > 0$ și un $n_0 \in \mathbb{N}_{>0}$ astfel încât:

$$P(n) : c_1 \cdot n \cdot log_2(n) \leq T(n) \leq c_2 \cdot n \cdot log_2(n), \forall n \geq n_0$$

1. **Caz de bază**

   Nu putem să alegem $n_0 = 1$ (deoarece $T(1) \neq 0$). Încercăm să demonstrăm pentru $n_0 = 2$.

   $$P(2) : c_1 \cdot 2 \cdot log_2(2) \leq T(2) \leq c_2 \cdot 2 \cdot log_2(2)$$

   Cu alte cuvinte, trebuie să arătăm că există $c_1$ și $c_2$ astfel încât: $2 \cdot c_1 \leq T(2) \leq 2 \cdot c_2$.

   Dacă înlocuim $T(2) = 2 \cdot T(1) + 2 \cdot c$, expresia devine:

   $$2 \cdot c_1 \leq 2 \cdot T(1) + 2 \cdot c \leq 2 \cdot c_2 \iff c_1 \leq T(1) + c \leq c_2$$

   $$c_1 \leq 1 + c \leq c_2 \iff \begin{cases} c_1 - 1 \leq c \\ c_2 - 1 \geq c \end{cases}$$

Știind valoarea c fixată din definiția lui T, putem opta pentru orice constantă $c_1$, respectiv $c_2$ care respectă relația anterioară.

**Este important însă ca la pasul de inducție să păstrăm alegerea făcută (să folosim aceleași constante)**.

2. **Pasul de inducție** $P(\frac{n}{2}) \Rightarrow P(n)$

   Presupunem că $P(m)$ adevărată, pentru $\forall m \in \{2, 4, \ldots, \frac{n}{2}\}$. Pornim de la:

$$c_1 \cdot \frac{n}{2} \cdot \log_2(\frac{n}{2}) \leq T(\frac{n}{2}) \leq c_2 \cdot \frac{n}{2} \cdot \log_2(\frac{n}{2})$$

   Înmulțim inegalitatea cu 2 și adunăm c · n:

$$c_1 \cdot n \cdot \log_2(\frac{n}{2}) + c \cdot n \leq 2 \cdot T(\frac{n}{2}) + c \cdot n \leq c_2 \cdot n \cdot \log_2(\frac{n}{2}) + c \cdot n$$

$$c_1 \cdot n \cdot (\log_2(n) - 1) + c \cdot n \leq T(n) \leq c_2 \cdot n \cdot (\log_2(n) - 1) + c \cdot n$$

$$c_1 \cdot n \cdot \log_2(n) - c_1 \cdot n + c \cdot n \leq T(n) \leq c_2 \cdot n \cdot \log_2(n) - c_2 \cdot n + c \cdot n$$

$$c_1 \cdot n \cdot \log_2(n) + n \cdot (c - c_1) \leq T(n) \leq c_2 \cdot n \cdot \log_2(n) + n \cdot (c - c_2)$$

   Pentru a demonstra $P(n)$ trebuie să arătăm:

$$c_1 \cdot n \cdot \log_2(n) \leq c1 \cdot n \cdot \log_2(n) + n \cdot (c - c_1) \leq T(n) \leq c_2 \cdot n \cdot \log_2(n) + n \cdot (c - c_2) \leq c_2 \cdot n \cdot \log_2(n)$$

   Cu alte cuvinte, inegalitatea este adevărată pentru orice $c_1$ și $c_2$ dacă: $c_1 \leq c \leq c_2$. În particular, putem alege $c_1 = c$ și $c_2 = c + 1$, astfel încât să respectăm toate constrângerile. Conform ipotezei de inducție, rezultă că $P(n)$ este adevărată pentru $n = 2^p, p \geq 1$.

   Rezultă că $T(n) \in \Theta(n \cdot \log(n))$. [1]

   Pentru mai multe detalii citiți 5.1

[1] Deși am analizat doar cazul când $n = 2^p, p \geq 1$, demonstrația nu își pierde din generalitate, vezi [6, p. 65], "In the case where $n$ is not a power of $c$, one can usually embed a problem of size $n$ in a problem of size $n'$, where $n'$ is the smallest power of $c$ equal to or greater than $n$.".

# 3 Teorema Master

Teorema Master este o metodă generală de rezolvare a anumitor tipuri de recurențe. Putem afirma că teorema Master ne oferă o "rețetă" de rezolvare a recurențelor, însă dezavantajul constă in faptul că ea nu este aplicabilă in orice situație.

## 3.1 Enunțul teoremei

Fie o recurență de forma $T(n) = aT(n/b) + f(n)$, unde $a \geq 1, b \gneq 1$ constante și $f(n)$ o funcție asimptotic crescătoare pozitivă. Putem afirma că T(n) aparține unei anumite clase de complexitate, in următoarele cazuri:

1. Dacă $f(n) \in O(n^{\log_b a - \epsilon})$, pentru o constantă $\epsilon > 0$, atunci $T(n) \in \Theta(n^{\log_b a})$.

2. Dacă $f(n) \in \Theta(n^{\log_b a} \log_2^k n)$, cu $k \geq 0$, atunci $T(n) \in \Theta(n^{\log_b a} \log_2^{k+1} n)$.

3. Dacă $f(n) \in \Omega(n^{\log_b a + \epsilon})$, pentru o constantă $\epsilon > 0$ și $af(n/b) \leq cf(n)$ pentru o constantă $c < 1$ și pentru un n suficient de mare, atunci $T(n) \in \Theta(f(n))$.

Pentru a ne da seama in ce caz ne aflăm, este recomandat să comparăm $f(n)$ cu $n^{\log_b a}$:

- dacă $f(n)$ este **polinomial mai mic** decat $n^{\log_b a} \Rightarrow$ caz 1 [2]
- dacă $f(n)$ este **în aceeași clasă de complexitate cu** $n^{\log_b a}$ (eventual inmulțit cu un factor logaritmic) $\Rightarrow$ caz 2
- dacă $f(n)$ este **polinomial mai mare** decat $n^{\log_b a} \Rightarrow$ caz 3

## 3.2 Math cheatsheet

- $\sum_{k=1}^{n} k = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$
- $\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3}$
- $\sum_{k=1}^{n} k^3 = \frac{n^2(n+1)^2}{4} \approx \frac{n^4}{4}$
- $\sum_{k=1}^{n} k^\alpha \approx \frac{n^{\alpha+1}}{\alpha+1}$
- $x_{n+1} = qx_n \Rightarrow \sum_{k=1}^{n} x_k = x_0 \frac{q^{n+1}-1}{q-1}$

Pentru exemple în plus ale aplicării Teoremei Master consultați link-ul [4] din Referințe.
Pentru un cheatsheet mai elaborat consultați link-ul [5] din Referințe.

[2] Acestă afirmație necesită ca $f(n)$ să fie asimptotic mai mică decât $n^{\log_b a}$ cu un factor de $n^\epsilon$, pentru o constantă $\epsilon \gneq 0$. [1, cap. 4.5, p. 94-95] Deci condiția, $f(n) \in o(n^{\log_b a})$, este necesară, dar nu suficientă.

# 4 Exerciții

**1.** Arătați că $T(n) = 2T(n/2) + \Theta(n) \in \Theta(n + n\log_2 n)$.

**2.** Rezolvați următoarele recurențe:

- **(a)** $T(n) = 3T(2n/3) + n^3 \log_2 n$
- **(b)** $T(n) = T(n/4) + 1$
- **(c)** $T(n) = 4T(n/2) + n\sqrt{n}$
- **(d)** $T(n) = 16T(n/4) + n!$
- **(e)** $T(n) = 8T(n/3) + 2^n$
- **(f)** $T(n) = T(n/4) + lgn$
- **(g)** $T(n) = 4T(n/2) + \frac{n}{lgn}$
- **(h)** $T(n) = 3T(n/3) + nlgn$
- **(i)** $T(n) = T(n/2 - \log_2 n) + 1$

**3.** Demonstrați că $T(n) = 5T(n/4) + n \in \Theta(5n^{\log_4 5} - 4n)$

**4.** Studiați posibilitatea aplicării teoremei Master in următoarele cazuri:

- **(a)** $T(n) = 2T(n/2) + n\log_2 n$
- **(b)** $T(n) = 3T(n/3) + \frac{n}{\log_2 n}$

**5.** Rezolvați următoarele recurențe:

- **(a)** $T(n) = 2T(n/2) + 1$
- **(b)** $T(n) = 3T(n/3) + n/2$
- **(c)** $T(n) = 3T(n/3) + n\log_2^2 n$
- **(d)** $T(n) = 3T(n/4) + n\log_2 n$
- **(e)** $T(n) = 5T(n/4) + n$

# 5 Anexe

## 5.1 Completarea demonstrației analizei complexității Merge-Sort

Întrucât materia acestui seminar nu își propune formalizarea fiecărui pas al demonstrației, și nici vouă **NU** o să vi se ceară subtilități când veți fi evaluați, am ales să scriem aici pentru cei interesați tratarea cazului $n \neq 2^p, p \in \mathbb{N}_{>0}$

Recurența care modelează numărul de pașii efectuați de algoritmul 1, indiferent de dimensiunea problemei, este:

$S(n) = S(\lfloor n/2 \rfloor) + S(\lceil n/2 \rceil) + c \cdot n, \forall n \in \mathbb{N} \wedge n \geq 2, c > 0, S(1) = 1$

Fie $k \in \mathbf{N}$, a.î. $2^k \leq n < 2^{k+1}$. Atunci: $\frac{n}{2} < 2^k \leq n < 2^{k+1} \leq 2 \cdot n$.

Observație: $\frac{n}{2}$ poate să fie o valoare fracționară deoarece începem să lucrăm cu funcții

Pentru $\forall n = 2^p, p \in \mathbb{N}_{>0}, S(n) = T(n)$. Vom presupune $S$ strict crescătoare, ceea ce vom și demonstra mai târziu.

$$\Rightarrow S(2^k) \leq S(n) < S(2^{k+1}) \iff T(2^k) \leq S(n) < T(2^{k+1})$$

Din cealaltă inducție:

$$c_1 \cdot 2^k \cdot \log_2(2^k) \leq S(n) < c_2 \cdot 2^{k+1} \cdot \log_2(2^{k+1})$$

Dar precum $\frac{n}{2} < 2^k, 2^{k+1} \leq 2 \cdot n$:

$$\Rightarrow c_1 \cdot \frac{n}{2} \cdot \log_2(\frac{n}{2}) \leq S(n) < c_2 \cdot 2 \cdot n \cdot \log_2(2 \cdot n)$$

$$\Rightarrow \frac{c_1}{2} \cdot n \cdot \log_2(n) - \frac{c_1}{2} \cdot n \leq S(n) < c_2 \cdot 2 \cdot n + c_2 \cdot 2 \cdot n \cdot \log_2(n)$$

Vom folosi inegalitatea: $n \leq \frac{n}{2} \cdot \log_2(n), \forall n \geq 4$.

Putem găsi o expresie mai mică decât termenul din stânga înlocuindu-l pe $\frac{c_1}{2} \cdot n$ cu $\frac{c_1}{4} \cdot n \cdot \log_2(n)$, inegalitate adevărată pentru un n suficient de mare. Analog, pe $c_2 \cdot 2 \cdot n$ îl putem înlocui cu $c_2 \cdot 2 \cdot n \cdot log(n)$.

$$\Rightarrow \frac{c_1}{4} \cdot n \cdot \log_2(n) \leq S(n) < 4 \cdot c_2 \cdot n \cdot \log_2(n), \forall n \geq n_0, n_0 \text{ fixat natural}$$

Reamintindu-ne definiția clasei $\Theta$, am găsit două constante, $c' = \frac{c_1}{4}$ și $c'' = 4c_2$, astfel încât $n \cdot \log_2(n)$ este o margine asimptotic tare a lui $S(n)$.

$$\Rightarrow S(n) = \Theta(n \cdot \log(n))$$

A mai rămas să demonstrăm că $S$ e o funcție strict crescătoare. Vom face asta prin inducție, demonstrând proprietatea:

$$Q(n) : T(n-1) \leq T(n), \forall n \in \mathbb{N}$$

**Baza inducției:** $S(1)$ o constantă. $S(2) = 2S(1) + 2c$. Avem $T(1) < T(2)$, deci $Q(2)$ adevărată.
**Pasul inducției:** Presupunem $Q(h), \forall h < n$ adevarată. Arătăm că: $Q(n-1) \rightarrow Q(n)$.

**Caz 1** Dacă $n$ este par, $n = 2k, k \in \mathbb{N}$.

$$Q(n) \Leftrightarrow S(n-1) \leq S(n)$$

$$Q(n) \Leftrightarrow S(k-1) + S(k) + c(2k-1) \leq 2S(k) + c(2k)$$

Ceea ce este adevarat din ipoteza inductivă și pentru că $c > 0$.

**Caz 2** Daca $n$ este impar, avem $n = 2k+1, k \in \mathbb{N}$.

$$Q(n) \Leftrightarrow S(n-1) \leq S(n)$$

$$Q(n) \Leftrightarrow 2S(k) + c(2k) \leq S(k) + S(k+1) + c(2k+1)$$

$$Q(n) \Leftrightarrow S(k) \leq S(k+1) + c$$

Ceea ce este adevarat din ipoteza inductivă și pentru că $c > 0$.

Astfel, am demonstrat că $Q(n)$ este adevărată, $\forall n \in \mathbb{N}$.

Vă recomandăm să citiți și alte demonstrații, precum [2] și [3].

# Referințe


[1] Cormen, T.H.; Leiserson, C.E.; Rivest, R.L.; Stein, C.: Introduction to Algorithms. 3rd edn. MIT Press and McGraw-Hill, USA (2009)

[2] CS 3110 Lecture 19, Substitution method for recurrence relations, `https://www.cs.cornell.edu/courses/cs3110/2008fa/lectures/lec19.html`. Ultima accesare la 6 Noi 2021

[3] Open Data Structures, Chap. 11.1, Comparison-Based Sorting, `http://opendatastructures.org/versions/edition-0.1e/ods-java/11_1_Comparison_Based_Sorti.html`. Ultima accesare la 6 Noi 2021

[4] Master Theorem: Practice Problems and Solutions, `https://www.csd.uwo.ca/~mmorenom/CS433-CS9624/Resources/master.pdf`. Ultima accesare la 6 Noi 2021

[5] Algorithm Analysis – Math CheatSheet, `https://raw.githubusercontent.com/AAResources/resources/main/Math_Cheatsheet___Algorithm_Analysis.pdf`. Ultima accesare la 6 Noi 2021

[6] Aho, Alfred V and Hopcroft, John E: The design and analysis of computer algorithms, Pearson Education India, 1974.